

27 // POP  
void pop() {  
if (top == -1) {  
cout << "underflow";  
return;  
}  
cout << arr[top] << " removed";  
top--;  
}

// PEEK  
void peek() {  
if (top == -1) {  
cout << "empty";  
return;  
}  
cout << "element" << arr[top];  
}

// DISPLAY  
void display() {  
if (top == -1) {  
cout << "empty";  
return;  
}  
for (int i = top; i >= 0; i--) {  
cout << arr[i];  
}

moto g57 power

28  
lex-2 Browser Back Button  
lex-3 Function Call Stack  
\* Stack using Collection Framework -  
java.util.Stack;  
-> Stack<Integer> stack = new Stack<>();  
stack.push(10);  
stack.push(20);  
stack.pop();  
cout << stack.peek();

\* Time complexity O(1)  
\* Stack using linkedlist  
\* Problems to do in Stack -  
- Reverse String  
- Balanced parentheses  
- Undo Feature  
- Browser History

NOVEMBER 20

SATURDAY  
OCT-25  
25 => Not in place Sorting:-  
i) Merge Sort  
ii) Counting Sort  
iii) Radix Sort

\* Stack  
-> Linear Data Structure  
-> Follows LIFO principle  
-> Operation happens from one end (TOP) only.

\* Where its using -  
Browsers - Back Button  
IDE - Undo/Redo  
OS - Function calls -

\* Stack Operations -  
- push - insert element  
- pop - remove top  
- peek()/top() - view top  
- isEmpty() - check empty  
- size() - count elements

g57 power

SUNDAY  
OCT-26  
26  
\* Stack internally stores data using  
- array  
- linkedlist

\* Rules -  
i) Push at TOP  
ii) Pop from TOP  
iii) Overflow if full  
iv) Underflow if empty

\* Implementation -  
class Stack {  
int arr[] = new int[5];  
int top = -1;  
// PUSH  
void push(int value) {  
if (top == arr.length - 1) {  
cout << "Overflow";  
return;  
}  
top++;  
arr[top] = value;  
cout << value << "Inserted";  
}

NOVEMBER 20

THURSDAY  
OCT-25  
23

```
static void radixSort(int[] arr) {
    int max = arr[0];
    for (int num : arr) {
        if (num > max) {
            max = num;
        }
    }
    for (int exp = 1; max/exp > 0; exp *= 10) {
        countingSort(arr, exp);
    }
}

public static void main(String[] args) {
    int[] arr = {170, 45, 75, 90, 802, 2};
    radixSort(arr);
}
```

Ex- [UPI transaction ID  
Ex- Aadhaar no. processing  
Ex- Railway seat numbers]

moto g57 power

Time complexity:  
 $O(d \times (n+k))$   
d = digit count  
n = no. of element  
k = range (0-9)

Stability and in-place Sorting:-  
- A sorting algo. is stable if equal elements maintain original relative order after sorting.  
→ Unstable Sorting algorithms-  
i) Heap sort  
ii) Quick sort  
iii) Selection sort  
→ In-place Sorting -  
Algorithm that uses very little extra memory - Ex- O(1)  
- No big extra arrays created

TUESDAY  
OCT-25  
21

Internal working:-

- Extract digit
  - Group numbers
  - Arrange them in place to next digit

A For extracting digits:-  
 $(arr[i] / exp) \% 10$

- unit digit -  
 $(arr[i] / 1) \% 10$
- tens digit  
 $(arr[i] / 10) \% 10$
- Hundreds digit  
 $(arr[i] / 100) \% 10$

2) static void countingSort(int[] arr, int exp) {

```
int n = arr.length;
int[] output = new int[n];
int[] count = new int[10];
```

moto g57 power

22

1) count digits

```
for (int i = 0; i < n; i++) {
    int digit = (arr[i] / exp) \% 10;
    count[digit]++;
}
```

2) Prefix sum

```
for (int i = 1; i < 10; i++) {
    count[i] += count[i-1];
}
```

3) Build output array

```
for (int i = n-1; i >= 0; i--) {
    int digit = (arr[i] / exp) \% 10;
    output[count[digit]-1] = arr[i];
    count[digit]--;
}
```

4) copy Back

```
for (int i = 0; i < n; i++) {
    arr[i] = output[i];
}
```



SUNDAY  
OCT 25

19

WK-42 - 2020-072

// create count array -

```
int count[] = new int[max+1];
```

// store frequencies :-

```
for (int i=0; i<arr.length; i++) {
    count[arr[i]]++;
}
```

int index = 0;

// Build sorted array :-

```
for (int i=0; i<count.length; i++) {
```

```
    while (count[i] > 0) {
```

```
        arr[index] = i;
```

```
        index++;
```

```
        count[i]--;
```

```
    }
```

```
}
```

public static void main (String[] args)

```
int arr[] = {4, 3, 1, 2, 9, 3, 2};
```

```
combinSort (arr);
```

```
for (int i=0; i<arr.length; i++) {
```

```
    System.out.print (arr[i] + " ");
```

```
}
```

moto g57 power

OCTOBER 2025

MONDAY  
OCT 26

20

WK-43 - 2020-073

\* Radix Sort :-

- Non-comparison sorting algo  
- Digits-by-digit sorting technique

- When numbers are large  
It sorts numbers from -

- Units place
- Tens place
- Hundreds place

→ Radix sort is faster than Merge sort and quick sort.

Ex - Banking - Transaction IDs, Aadhaar no., Mobile no.

Ex - 170, 45, 175, 90, 802, 24, 2, 166

i) unit digit

170, 90, 802, 2, 24, 45, 175, 166

ii) Tens digit

802, 2, 24, 45, 66, 170, 75, 90

iii) Hundreds digit

2, 24, 45, 66, 75, 90, 170, 802

NOVEMBER

Mon Tue Wed Thu Fri Sat Sun Mon Tue Wed Thu Fri Sat Sun

FRIDAY  
OCT 25

17

WK-42 - 2020-072

```
int[] heap;
```

```
int size;
```

```
int capacity;
```

```
void minHeap (int capacity) {
```

```
    this.capacity = capacity;
```

```
    heap = new int[capacity];
```

```
    size = 0;
```

```
}
```

```
void insert (int value) {
```

```
    heap[size] = value;
```

```
    int current = size;
```

```
    while (current > 0 && heap[current] <
```

```
        heap[(current-1)/2]) {
```

```
        int temp = heap[current];
```

```
        heap[current] = heap[(current-1)/2];
```

```
        heap[(current-1)/2] = temp;
```

```
        current = (current-1)/2;
```

```
    }
```

```
    size++;
```

moto g57 power

OCTOBER 2025

SATURDAY  
OCT 26

18

WK-43 - 2020-073

```
void display () {
```

```
    for (int i=0; i<size; i++) {
```

```
        System.out.print (heap[i] + " ");
```

```
    }
```

\* Counting Sort :-

Non-comparison sorting

algorithm -

- uses counting / frequency technique

Working :-

i) Find maximum

ii) Create count array

iii) Count frequency

iv) Build sorted array

Ex -

```
static void countingSort (int arr[]) {
```

```
    int max = arr[0];
```

```
    // find max element
```

```
    for (int i=1; i<arr.length; i++) {
```

```
        if (arr[i] > max) {
```

```
            max = arr[i];
```

```
        }
```

```
    }
```

NOVEMBER

Mon Tue Wed Thu Fri Sat Sun Mon Tue Wed Thu Fri Sat Sun

WEDNESDAY

OCT-25

15

WK-42 - 2023-24

static void heapsort(int arr[])

int n = arr.length;

// Build Max Heap;

for (int i = n/2 - 1; i &gt;= 0; i--)

heapify(arr, n, i);

// Extract elements one by one -

for (int i = n - 1; i &gt; 0; i--)

int temp = arr[0];

arr[0] = arr[i];

arr[i] = temp;

heapify(arr, i, 0);

public static void main (String[] args)

int arr[] = {4, 10, 3, 5, 1};

OCTOBER 2023

moto g57 power

|    | Sun | Mon | Tue | Wed | Thu | Fri | Sat |
|----|-----|-----|-----|-----|-----|-----|-----|
| 1  |     |     |     |     |     |     |     |
| 2  |     |     |     |     |     |     |     |
| 3  |     |     |     |     |     |     |     |
| 4  |     |     |     |     |     |     |     |
| 5  |     |     |     |     |     |     |     |
| 6  |     |     |     |     |     |     |     |
| 7  |     |     |     |     |     |     |     |
| 8  |     |     |     |     |     |     |     |
| 9  |     |     |     |     |     |     |     |
| 10 |     |     |     |     |     |     |     |
| 11 |     |     |     |     |     |     |     |
| 12 |     |     |     |     |     |     |     |
| 13 |     |     |     |     |     |     |     |
| 14 |     |     |     |     |     |     |     |
| 15 |     |     |     |     |     |     |     |
| 16 |     |     |     |     |     |     |     |
| 17 |     |     |     |     |     |     |     |
| 18 |     |     |     |     |     |     |     |
| 19 |     |     |     |     |     |     |     |
| 20 |     |     |     |     |     |     |     |
| 21 |     |     |     |     |     |     |     |
| 22 |     |     |     |     |     |     |     |
| 23 |     |     |     |     |     |     |     |
| 24 |     |     |     |     |     |     |     |
| 25 |     |     |     |     |     |     |     |
| 26 |     |     |     |     |     |     |     |
| 27 |     |     |     |     |     |     |     |
| 28 |     |     |     |     |     |     |     |
| 29 |     |     |     |     |     |     |     |
| 30 |     |     |     |     |     |     |     |
| 31 |     |     |     |     |     |     |     |

heapsort(arr);

for (int num: arr)

System.out.println(num);

Ex- Banking priority system  
 Ex- Railway ticket priority  
 Ex- Shopping website top discount

\* Time complexity =  $O(n \log n)$ 

\* Min Heap :- Binary tree where parent node is smaller than children

Ex-1 Food delivery priority:- to nearest delivery partner

Ex-2 Railway platform scheduling

Ex-3 Online Exam Ranking

Time Complexity:-

Insert -  $O(\log n)$ Delete -  $O(\log n)$ get min -  $O(1)$ 

NOVEMBER 2023

|    | Sun | Mon | Tue | Wed | Thu | Fri | Sat |
|----|-----|-----|-----|-----|-----|-----|-----|
| 1  |     |     |     |     |     |     |     |
| 2  |     |     |     |     |     |     |     |
| 3  |     |     |     |     |     |     |     |
| 4  |     |     |     |     |     |     |     |
| 5  |     |     |     |     |     |     |     |
| 6  |     |     |     |     |     |     |     |
| 7  |     |     |     |     |     |     |     |
| 8  |     |     |     |     |     |     |     |
| 9  |     |     |     |     |     |     |     |
| 10 |     |     |     |     |     |     |     |
| 11 |     |     |     |     |     |     |     |
| 12 |     |     |     |     |     |     |     |
| 13 |     |     |     |     |     |     |     |
| 14 |     |     |     |     |     |     |     |
| 15 |     |     |     |     |     |     |     |
| 16 |     |     |     |     |     |     |     |
| 17 |     |     |     |     |     |     |     |
| 18 |     |     |     |     |     |     |     |
| 19 |     |     |     |     |     |     |     |
| 20 |     |     |     |     |     |     |     |
| 21 |     |     |     |     |     |     |     |
| 22 |     |     |     |     |     |     |     |
| 23 |     |     |     |     |     |     |     |
| 24 |     |     |     |     |     |     |     |
| 25 |     |     |     |     |     |     |     |
| 26 |     |     |     |     |     |     |     |
| 27 |     |     |     |     |     |     |     |
| 28 |     |     |     |     |     |     |     |
| 29 |     |     |     |     |     |     |     |
| 30 |     |     |     |     |     |     |     |
| 31 |     |     |     |     |     |     |     |

MONDAY

OCT-25

13

WK-42 - 2023-24

\* Heap sort

Heap is a special binary tree :- Max, Min Heap

Max Heap rule :- Parent should be greater than children.

\* For any index i:-

Left child :-  $2i + 1$ Right child :-  $2i + 2$ Parent :-  $(i - 1) / 2$ 

\* Interval working:-

i) Convert array into Max Heap  
- largest element to the root

ii) Swap root with last element

iii) Reduce heap size  
- ignore last sorted element

(iv) Heapify again

Ex- [4, 10, 3, 5, 1]

→ Build max heap - [10, 4, 3, 5, 1]

→ swap 10 with 1 - [1, 4, 3, 5, 10]

→ Heapify again - finally - [1, 3, 4, 5, 10]

moto g57 power

OCTOBER 2023

|    | Sun | Mon | Tue | Wed | Thu | Fri | Sat |
|----|-----|-----|-----|-----|-----|-----|-----|
| 1  |     |     |     |     |     |     |     |
| 2  |     |     |     |     |     |     |     |
| 3  |     |     |     |     |     |     |     |
| 4  |     |     |     |     |     |     |     |
| 5  |     |     |     |     |     |     |     |
| 6  |     |     |     |     |     |     |     |
| 7  |     |     |     |     |     |     |     |
| 8  |     |     |     |     |     |     |     |
| 9  |     |     |     |     |     |     |     |
| 10 |     |     |     |     |     |     |     |
| 11 |     |     |     |     |     |     |     |
| 12 |     |     |     |     |     |     |     |
| 13 |     |     |     |     |     |     |     |
| 14 |     |     |     |     |     |     |     |
| 15 |     |     |     |     |     |     |     |
| 16 |     |     |     |     |     |     |     |
| 17 |     |     |     |     |     |     |     |
| 18 |     |     |     |     |     |     |     |
| 19 |     |     |     |     |     |     |     |
| 20 |     |     |     |     |     |     |     |
| 21 |     |     |     |     |     |     |     |
| 22 |     |     |     |     |     |     |     |
| 23 |     |     |     |     |     |     |     |
| 24 |     |     |     |     |     |     |     |
| 25 |     |     |     |     |     |     |     |
| 26 |     |     |     |     |     |     |     |
| 27 |     |     |     |     |     |     |     |
| 28 |     |     |     |     |     |     |     |
| 29 |     |     |     |     |     |     |     |
| 30 |     |     |     |     |     |     |     |
| 31 |     |     |     |     |     |     |     |

static void heapify(int arr,

int n, int i)

Heap size current parent index

int largest = i; // parent element

int left =  $2 * i + 1$ ; // left child index

if (left &lt; n)

int right =  $2 * i + 2$ ;if (left < n && arr[left] > arr[largest])  
// left child comparison  
largest = left;if (right < n && arr[right] > arr[largest])  
// right child comparison  
largest = right;

if (largest != i) // swap and heapify

int temp = arr[i];

arr[i] = arr[largest];

arr[largest] = temp;

heapify(arr, largest, n);

TUESDAY

OCT-25

14

WK-42 - 2023-24

|    | Sun | Mon | Tue | Wed | Thu | Fri | Sat |
|----|-----|-----|-----|-----|-----|-----|-----|
| 1  |     |     |     |     |     |     |     |
| 2  |     |     |     |     |     |     |     |
| 3  |     |     |     |     |     |     |     |
| 4  |     |     |     |     |     |     |     |
| 5  |     |     |     |     |     |     |     |
| 6  |     |     |     |     |     |     |     |
| 7  |     |     |     |     |     |     |     |
| 8  |     |     |     |     |     |     |     |
| 9  |     |     |     |     |     |     |     |
| 10 |     |     |     |     |     |     |     |
| 11 |     |     |     |     |     |     |     |
| 12 |     |     |     |     |     |     |     |
| 13 |     |     |     |     |     |     |     |
| 14 |     |     |     |     |     |     |     |
| 15 |     |     |     |     |     |     |     |
| 16 |     |     |     |     |     |     |     |
| 17 |     |     |     |     |     |     |     |
| 18 |     |     |     |     |     |     |     |
| 19 |     |     |     |     |     |     |     |
| 20 |     |     |     |     |     |     |     |
| 21 |     |     |     |     |     |     |     |
| 22 |     |     |     |     |     |     |     |
| 23 |     |     |     |     |     |     |     |
| 24 |     |     |     |     |     |     |     |
| 25 |     |     |     |     |     |     |     |
| 26 |     |     |     |     |     |     |     |
| 27 |     |     |     |     |     |     |     |
| 28 |     |     |     |     |     |     |     |
| 29 |     |     |     |     |     |     |     |
| 30 |     |     |     |     |     |     |     |
| 31 |     |     |     |     |     |     |     |



SATURDAY  
OCT 25

11

WK-41 - 284-081

```
static void quicksort (int[] arr,
                      int start, end, int end?)
{
    if (start < end) {
        int first = partition (arr,
                               start, end);
        quicksort (arr, start, first - 1);
        quicksort (arr, first + 1, end);
    }
}
```

```
static int partition (int[] arr,
                     int start, int end) {
    int first = arr[end];
    int i = start - 1;
    for (int j = start; j < end; j++) {
        if (arr[j] < first) {
            i++;
        }
    }
}
```

moto g57 power

SUNDAY  
OCT 25

12

WK-41 - 285-080

```
int temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;

int temp = arr[i+1];
arr[i+1] = arr[end];
arr[end] = temp;

return i+1;
```

Ex-1 Need smaller transaction first  
500, 100, 600, 300

Ex-2 Railway ticket prices

Ex-3 student marks

\* Best case -  $O(n \log n)$

Worst case -  $O(n^2)$

THURSDAY  
OCT 25

09

WK-41 - 282-083

\* Quick Sort:- Recursive algo used to arrange elements in asc. or desc. order. Divide and conquer. 50, 40, 30, 10, 20

Working:- choose first

Partition  
(smaller element-left) → [20, 10]  
(bigger element-right) → [50, 40]

Again sort  
left and right part 10, 20, 30, 40, 50

Variables - start, end, first, i (smaller elements tracker), j (bigger)

ex- [7, 2, 9, 1, 5]

start=7, end=01 pivot=5

i) move start and end, compare and swap

\* start will stop moving if arr[i] > pivot  
end will stop moving if arr[j] < pivot

→ 7 > 5 yes → swap  
→ start will point to 7

moto g57 power

FRIDAY  
OCT 25

10

WK-41 - 283-082

end < pivot → yes.  
end = 1  
Now swap 1 and 7.

→ 1, 2, 9, 7, 5.

start=1 pivot=5 end=7, pivot=5

i) 1 > 5 no, start++  
ii) 2 > 5 no, start++  
iii) 9 > 5 yes start=9  
iv) 7 < 5 no, end--  
v) 5 < 5 yes end=2

swap 9 and 5 since start and end crossed each other, place pivot = arr[start]

→ 1, 2, 5, 9, 7.

Now recursively do with left and right array.

after: 1, 2, 5, 7, 9.

TUESDAY

OCT 25

07

WE-41 - 290-088

```

int[] L = new int[n1];
// temp left array created

int[] R = new int[n2];
// temp right array created

for (int i = 0; i < n1; i++)
// loop for copying element
L[i] = arr[start + i];

for (int j = 0; j < n2; j++)
R[j] = arr[mid + 1 + j];

int i = 0, j = 0, k = start;
while (i < n1 && j < n2)
{
    if (L[i] <= R[j])
    {
        arr[k] = L[i];
        i++;
    }
    else
    {
        arr[k] = R[j];
        j++;
    }
    k++;
}

while (i < n1)
    arr[k++] = L[i];

while (j < n2)
    arr[k++] = R[j];

```

moto g57 power

WEDNESDAY

OCT 25

08

WE-41 - 290-088

```

while (i < n1)
    arr[k++] = R[j++];

public static void main(String[] args)
{
    int arr[] = {8, 3, 1, 5};
    mergeSort(arr, 0, arr.length - 1);
    for (int num : arr)
        print(num + " ");
}

Ex-1 IITC sorting passenger ID
Ex-2 Online shopping filter
Ex-3 Banking transactions

> Best case  $O(n \log n)$ 
Worst case  $O(n \log n)$ 

```

SUNDAY

OCT 25

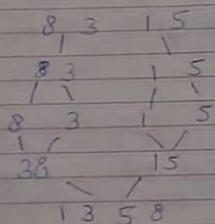
05

WE-41 - 278-087

Merge Sort :-

Divide and conquer sorting algorithm that-

divides array into halves  
 recursively sorts halves  
 merge sorted halves



Algo- Divide array  
 ↓  
 Merge sort left half  
 ↓  
 sort right half  
 Merge sorted array

moto g57 power

MONDAY

OCT 25

06

WE-41 - 279-088

```

// static void mergeSort (int[] arr,
// int start, int end)
// start      end
//
// if (start < end) // check no. of element in arr
// {
//     int mid = (start + end) / 2;
//     mergeSort(arr, start, mid);
//     // sort left half recursively
//     mergeSort(arr, mid + 1, end);
//     // sort right half recursively
//     merge(arr, start, mid, end);
//
//     // combine sorted left and right
//     // half into one sorted arr
// }
//
// static void merge (int[] arr,
// int start, int mid, int end)
// {
//     int n1 = mid - start + 1;
//     // calculate left array size
//     int n2 = end - mid;
//     // calculate right array size
// }

```

NOVEMBER 2023



FRIDAY  
OCT 25  
03

\* Insertion Sort:-  
Comparison-based sorting algo.

Pick element  
↓  
Compare left side  
↓  
Shift bigger element  
↓  
Insert correctly  
↓  
Repeat

\* Syntax:-  
for (int i = 1; i < n; i++) {  
    int current = arr[i];  
    int j = i - 1;  
    while (j > 0 && arr[j] > current) {  
        arr[j + 1] = arr[j];  
        j--;  
    }  
    arr[j + 1] = current;  
}

moto g57 power

SATURDAY  
OCT 25  
04

Ex-1 Movie Ticket prices:-

```
int[] prices = {500, 300, 100, 200};
for (int i = 1; i < prices.length; i++) {
    int current = prices[i];
    int j = i - 1;
    while (j > 0 && prices[j] > current) {
        prices[j + 1] = prices[j];
        j--;
    }
    prices[j + 1] = current;
}
for (int price : prices) {
    cout << price << " ";
}
```

Ex-2 Adhaar Queue Numbers

Ex-3 Student Marks Sorting

Worst Case -  $O(n^2)$  - reverse sorted array  
Best Case -  $O(n)$  - already sorted

NOVEMBER 2021

WEDNESDAY  
OCT 25  
01

Ex-2 Shopping App Ratings

Ex-3 Office Emp. IDs

\* Worst case -  $O(n^2)$  - Reverse sorted array  
\* Best case -  $O(n)$  - already sorted

2) Selection Sort:-  
Comparison-based sorting algo.

→ Find minimum element  
↓  
swap with current index  
↓  
move forward  
↓  
Repeat

\* Syntax:-  
for (int i = 0; i < n - 1; i++) {  
    int min = i;  
    for (int j = i + 1; j < n; j++) {  
        if (arr[j] < arr[min]) {  
            min = j;  
        }  
    }

moto g57 power

THURSDAY  
OCT 25  
02

Ex-1 ATM transaction Sorting

```
int[] amounts = {1000, 1000, 3000, 2000};
for (int i = 0; i < amounts.length - 1; i++) {
    int min = i;
    for (int j = i + 1; j < amounts.length; j++) {
        if (amounts[j] < amounts[min]) {
            min = j;
        }
    }
    int temp = amounts[i];
    amounts[i] = amounts[min];
    amounts[min] = temp;
}
for (int amount : amounts) {
    cout << amount << " ";
}
```

Ex-2 Adhaar Queue token Sorting

Ex-3 Office file number Sorting

Best Case -  $O(n^2)$   
Worst Case -  $O(n^2)$

NOVEMBER 2021

1. Railway ticket prices:-

500, 300, 100, 200

2) `int[] prices = {500, 300, 100, 200};`

`for (int i=0; i < prices.length-1; i++)`

`for (int j=0; j < prices.length-1-i; j++)`

`if (prices[j] > prices[j+1])`

`int temp = prices[j];`

`prices[j] = prices[j+1];`

`prices[j+1] = temp;`

`}}`

`for (int price : prices)`

`out (price);`

`}}`

i) `j=0, temp=500, 300`

`500 > 300`

swap and `temp=500, 300, 100, 200`

ii) `j=1, temp=500`

`500 > 100 No. 300, 100, 500, 200`

iii) `i=2, j=2, temp=100`

`100 > 200 yes.`

swap with `temp`, and `temp=100.`

`300, 100, 500, 200`

`100, 300, 500, 200`

SEPT 2023

| Sun | Mon | Tue | Wed | Thu | Fri | Sat | Sun |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   |
| 9   | 10  | 11  | 12  | 13  | 14  | 15  | 16  |
| 17  | 18  | 19  | 20  | 21  | 22  | 23  | 24  |
| 25  | 26  | 27  | 28  | 29  | 30  | 31  |     |



## N Binary search:-

divides sorted data repeatedly

Working:-

↓  
find middle↓  
compare targetAnswer  $\rightarrow$  go left  
larger  $\rightarrow$  go right

Ex- Railway seat search:-

int[] arr = {1, 10, 20, 30, 40, 50};

int target = 40;

int start = 0;

int end = arr.length - 1;

while (start &lt;= end) {

int mid = (start + end) / 2;

if (arr[mid] == target) {

return mid;

}

else if (target &gt; arr[mid]) {

start = mid + 1;

}

else {

end = mid - 1;

}

Ex-2 Customer ID search

int[] id = {1000, 200, 1300, 400};

int index = Arrays.binarySearch(id, 300);

Ex-3 Train Bank search

Time complexity =

Best case -  $O(1)$ Worst case -  $O(\log n)$

27

## \* Binary search:-

divides sorted data repeatedly.

Working:-

Find middle

compare target

Answer  $\rightarrow$  go left  
Bigger  $\rightarrow$  go right

Ex- Railway seat search:-

int[] arr = {1000, 30140150};

int target = 40;

int start = 0;

int end = arr.length - 1;

while (start &lt;= end) {

int mid = (start + end) / 2;

if (arr[mid] == target) {

return (mid);

break;

}

28

else if (target &gt; arr[mid]) {

start = mid + 1;

} else {

end = mid - 1;

}

Ex-2 Customer ID search

int[] id = {100, 200, 300, 400};

int index = Arrays.binarySearch(id, 300);

return (index);

Ex-3 Library Book search

\* Time complexity -

Best case -  $O(1)$ Worst case -  $O(\log n)$



THURSDAY

SEP-25

25

\* Traversal of LL:-

Node temp = first;

do {

cout << temp->data;

temp = temp->next;

} while (temp != first);

\* Searching:-

Finding required element from collection of data.

1) Linear search 2) Binary search

checks the element at a time sequentially

Best for:- small datasets

- traversed arrays

- simple searching

Algorithm:-

for (int i=0; i<arr.length; i++) {

if (arr[i] == target) {

found

}

}

FRIDAY

SEP-25

26

Ex-1 UPI transaction search -

int[] transaction = {101, 104, 103, 104, 5};

int target = 103;

boolean found = false;

for (int i=0; i<transaction.length; i++)

if (transaction[i] == target) {

found = true;

break;

Ex-2 Array verification

- 1 means element not found

Ex-3 shopping app product search

int[] prices = {500, 100, 300};

int target = 100;

for (int price : prices) {

if (price == target) {

cout << "found";

Time complexity -

Best case -  $O(1)$

Worst case -  $O(n)$

OCTOBER 2023

SEP 25

SEP 26

SEP 27

SEP 28

SEP 29

SEP 30

OCT 1

OCT 2

OCT 3

OCT 4

OCT 5

OCT 6

OCT 7

OCT 8

OCT 9

OCT 10

OCT 11

OCT 12

OCT 13

OCT 14

OCT 15

OCT 16

OCT 17

OCT 18

OCT 19

OCT 20

OCT 21

OCT 22

OCT 23

OCT 24

OCT 25

OCT 26

OCT 27

OCT 28

OCT 29

OCT 30

OCT 31

## \* Doubly linked list

Each node contains -

- previous
- data
- next

null  $\leftarrow$  10  $\leftrightarrow$  20  $\leftrightarrow$  30  $\rightarrow$  null

→ DLL supports -

- Forward traversal
- Backward traversal

## \* DLL creation :-

Print data;

Node prev;

Node next;

Node first = new Node(10);

Node second = new Node(20);

first.next = second;

second.prev = first;

## \* DLL operations -

Insert

update first

delete

searched both side

traverse forward

next

traverse backward

prev

## \* Circular linked list :-

10  $\rightarrow$  20  $\rightarrow$  30 $\uparrow$  $\leftarrow$  10  $\leftarrow$  20  $\leftarrow$  30  $\leftarrow$ 

used in -

- CPU scheduling
- Round Robin scheduling

## \* Creation of CLL :-

Node first = new Node(10);

Node second = new Node(20);

Node third = new Node(30);

first.next = second;

second.next = third;

third.next = first;



21

(ii) delete last node :-

Node (int data) ?  
this->data = data;

Node head = new Node(10);

head  
[10, null]

head->next = new Node(20);

head  
↓

[10, 20] → [20, null]

head->next->next = new Node(30);

[10, 20] → [20, 30] → [30, null]

Node temp = head;

while (temp->next->next != null)

temp->next->next = null; // delete last node

Iteration 1

temp = 10  
next = 30 ? = null?

Iteration 2  
temp = 20  
next = null? = null  
temp->next->next = null  
temp stop

temp = 30  
next = null

Before 20 → 30  
Now 20 → null

22

Searching - operation :-

Node temp = head;

int target = 20;

while (temp->next != null)

if (temp->data == target)

cout << " ";

}

temp = temp->next

\* Reverse linked list

Node prev = null;

Node current = head;

Node next = null;

while (current->next != null)

next = current->next;

current->next = prev;

prev = current;

current = next;

}

head = prev

OCTOBER 2025

19

head.next = new Node(20);

head

[10 | .] → [20 | null]

Node temp = head;

while (temp.next != null) {  
temp = temp.next; }

temp.next = new Node(30);

// Inserted at end.

temp = head; // traversal again  
while (temp != null)  
{  
cout << temp.data << " ";  
}

at Middle :-

Node head = new Node(10);  
head.next = new Node(30);

head

[10 | .] → [30 | null]

Node new Node = new Node(10);  
new Node.next = head.next;  
add. of node 30

20

Before 10 → 30 → null  
20 → null

Now 10 → 30,  
20 → 30

head.next = new Node;

Now 10 → 20 → 30 → null  
Node temp = head;

\* Deletion operation :-

(i) Deletion first node O(1)

Node head = new Node(10)  
Node new head.next = new Node(20);

head

[10 | .] → [20 | null]

head = head.next; // delete

[10 | .] → [20 | .]

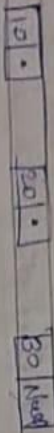
by unacademy, are delete later

OCTOBER 2023



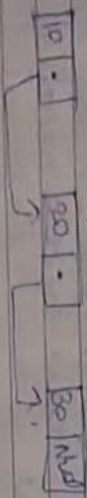
Node first = new Node(10);  
Node second = new Node(20);  
Node third = new Node(30);

// Spawnted value in Node



first.next = second;  
second.next = third;

// Connected nodes



Node temp = first;

while (temp != null);

cout (temp.data);

temp = temp.next;

// traverse list

## Operations on Singly LL:-

- Insertion
- Deletion
- Updating
- Reversing

1) Insertion :- Moving node by node

Node temp = head;

while (temp != null);  
cout (temp.data);  
temp = temp.next;

2) Insertion → (Beginning  
→ End  
→ Middle

i) at Beginning

Node head = new Node(10);  
Node newNode = new Node(130);  
newNode.next = head;  
head = newNode;

(ii) at end

Node head = new Node(10);  
Node newNode = new Node(130);  
newNode.next = null;  
head = newNode;

⇒ head → 10 → null

## linked list

Collection of nodes connected using references (links)

Each node contains -  
• data  
• address of next node.

|    |    |    |    |    |      |
|----|----|----|----|----|------|
| 1  | 30 | 2  | 40 | 3  | null |
| 00 |    | 30 |    | 40 |      |

It solves array limitations:-

i) data changes dynamically

ii) insertion/deletion fast

iii) memory allocation dynamic

Ex- Browser history, music playlist, operating systems

3) Node - single reference  
Head - first node  
Tail - last node  
Link - connection/reference

Types :-

- Singly LL
- Doubly LL
- Circular LL
- Circular doubly LL

Each node contains -

data + next node reference

\* Node Creation :-

```
int data;  
Node next;
```

```
Node (int data) {  
    this->data = data; //value  
    this->next = null; //address  
}
```

Ex- office token queue:-

```
?  
int data;  
Node next;
```

```
Node (int data) {  
    this->data = data;  
}
```

```
{  
    // create node
```

TUESDAY  
SEP 25

16

10:30 - 12:00 PM

OCTOBER

OCTOBER 2025

Mon Tue Wed Thu Fri Sat Sun  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31



## Array operation:-

i) Deletion:- <sup>one by one</sup> deleting elements

ex- VPI app showing transaction history

int transaction = 1100, 1200, 1300, 1400;

for (int i = 0; i < transaction.length; i++) {

system.out.println(transaction[i]);

ii) Insertion:- (fixed size  $\rightarrow$  shifting elements manually)

int position = 1;

int val = 15;

for (int i = array.length; i > position; i--)

{ array[i] = array[i-1];

array[position] = val;

## Deletion

ii) Searching - Linear search  
checks element one by one

- Binary search -

Array must be sorted  
 $O(\log n)$

(v) Updation

(vi) Sorting - arranging data  
Arrays.sort();

(vii) Reversing - Reverse loop

(viii) Copying - Arrays.copyOf (arr);

→

## 2D Arrays

great arrangement:-

int [] seats = {

{1, 2, 3},  
{4, 5, 6},

};

→ Traversal - for each loop

THURSDAY  
SEP 25

11

Java Arrays

Static life example -  
Classroom Register

→ Arrays used to store multiple values in one variable.

→ Array is a collection of similar type of data stored in contiguous memory location

Ex - Different heap in org.

→ heap 1

heap 2

heap 3

→ heap 1.

Array :-

int[] arr; or

int arr[];

Mem. allocation -

arr = new int[5]

or int arr[] = new int[5];

SEPTEMBER 2024

FRIDAY  
SEP 25

12

Initialization -  
int arr[] = {1, 0, 10, 30, 40};

Access here  
System.out.println(arr[0]);

for (int i = 0; i < arr.length; i++)  
System.out.println(arr[i]);

2D Array -  
int [][] arr = {  
    {1, 2, 3},  
    {3, 4, 5},  
};

Default values in Arrays -  
int 0  
double 0.0  
boolean false  
String null

→ Arrays are stored in heap memory

OCTOBER 2025

OCTOBER

NOVEMBER

DECEMBER